# REST API mit Python & FastAPI

## Von der User Story zum funktionsfähigen API

---

# Lernziele

## Was du heute lernen wirst (und warum du danach APIs liebst!)

Nach dieser Schulung wirst du:

### Nie wieder planlos APIs bauen

- Verstehen warum Pseudocode vor dem API-Code kommt
- Deine API-Logik strukturiert durchdenken können

### REST APIs mit System entwickeln

- User Stories in API-Endpoints übersetzen
- FastAPI effektiv nutzen für moderne APIs

### Professionelle Entwicklungsumgebung nutzen

- Docker Compose für einfache Datenbank-Setup
- Postman für systematisches API-Testing

### Von der Idee zur produktiven API

- Schritt-für-Schritt von Pseudocode zu echtem Code
- Best Practices von Anfang an anwenden

**Das Beste:** Du wirst verstehen warum geplante APIs stabiler und wartbarer sind!

---

# Das Problem kennen wir alle

## Warum scheitern so viele API-Projekte?

**Kennst du das?**

```
# Du startest voller Motivation...
@app.post("/users")
def create_user():
    # Hmm, welche Daten brauche ich eigentlich?
    # Ach ja, validieren... aber was genau?
    # Moment, wie speichere ich das in der DB?
    # Oh, und was ist mit Fehlern?
    # Und die API-Dokumentation?
    # HILFE! 😱
```

**Das Problem:** Du springst direkt in den Code ohne Plan!

**Das Ergebnis:**

- APIs die nicht richtig funktionieren
- Unverständliche Fehlermeldungen
- Chaotische Code-Struktur
- Stundenlange Debugging-Sessions
- Frustrierte Frontend-Entwickler

---

# Die Lösung: Pseudocode für APIs

## Denken BEVOR Programmieren

**Stell dir vor:** Du baust ein Haus ohne Bauplan. Wie wird das wohl ausgehen?

**Genauso ist API-Entwicklung ohne Pseudocode:**

- Du weißt nicht welche Endpoints du brauchst
- Du vergisst wichtige Validierungen
- Du verlierst dich in FastAPI-Syntax-Details
- Am Ende ist deine API unbrauchbar

**Mit Pseudocode passiert das nicht:**

> "Pseudocode ist dein Bauplan für APIs - erst denken, dann bauen!"

---

# Warum FastAPI perfekt für geplante APIs ist

## Keine Angst - du kennst schon das Wichtigste!

**Du hast schon Flask gelernt?** Perfekt! FastAPI ist wie Flask's moderner, hilfsbereiter großer Bruder.

### Was bleibt gleich:

- **Python Web-Entwicklung** - gleiche Grundprinzipien
- **HTTP-Methoden** (GET, POST, PUT, DELETE)
- **JSON-Handling** - nur einfacher
- **Routing-Konzept** - du erkennst alles wieder

### Was wird besser mit FastAPI:

- **Weniger Code** für das gleiche Ergebnis
- **Automatische Dokumentation** - deine API erklärt sich selbst
- **Automatische Validierung** - weniger Bugs durch Pydantic
- **Moderne Python-Features** (Type Hints) von Anfang an

### Warum das perfekt für Pseudocode ist:

FastAPI's Struktur entspricht genau dem wie wir in Pseudocode denken!

---

slideType: section

# Von der User Story zum API-Plan

---

# Unsere User Story für heute

## Ein konkretes, realistisches Beispiel

**Die User Story:**

> "Als Personalverantwortlicher möchte ich Mitarbeiter-Daten digital verwalten können, damit ich nicht mehr mit Excel-Tabellen arbeiten muss"

**Was bedeutet das konkret?**

- Neue Mitarbeiter anlegen
- Alle Mitarbeiter auflisten
- Einzelne Mitarbeiter anzeigen
- Mitarbeiter-Daten aktualisieren
- Mitarbeiter löschen (bei Kündigung)

**Das ist ein klassisches CRUD-System (Create, Read, Update, Delete)**

---

# User Story analysieren

## Was muss unsere API können?

### Schritt 1: Welche Aktionen brauchen wir?

- **Mitarbeiter erstellen** → POST Request
- **Alle Mitarbeiter auflisten** → GET Request
- **Einen Mitarbeiter anzeigen** → GET Request mit ID
- **Mitarbeiter aktualisieren** → PUT Request mit ID
- **Mitarbeiter löschen** → DELETE Request mit ID

### Schritt 2: Welche Daten haben Mitarbeiter?

- Name (Pflichtfeld)
- Email (Pflichtfeld, muss valide sein)
- Position/Rolle (optional)
- Startdatum (optional)
- Gehalt (optional, vertraulich)

### Schritt 3: Was kann schief gehen?

- Ungültige Eingabedaten
- Doppelte Email-Adressen

- Mitarbeiter existiert nicht
- Datenbank ist nicht erreichbar

---

# API-Endpoints planen

## Von Aktionen zu REST-Endpoints

### Unsere geplanten Endpoints:

```
GET    /employees      → Alle Mitarbeiter auflisten
POST   /employees      → Neuen Mitarbeiter erstellen
GET    /employees/{id} → Einen Mitarbeiter anzeigen
PUT    /employees/{id} → Mitarbeiter aktualisieren
DELETE /employees/{id} → Mitarbeiter löschen
```

### Das ist REST-konform:

- **Ressource:** `/employees` (Plural!)
- **Aktionen:** Über HTTP-Methoden definiert
- **Identifikation:** Über ID in der URL
- **Einheitlich:** Gleiches Muster für alle Aktionen

**Merkregel:** REST APIs sind wie Ordnerschränke - strukturiert und vorhersagbar!

---

# Pseudocode: Mitarbeiter erstellen

## Schritt für Schritt durchdenken

```
FUNCTION create_employee(employee_data):

  # Input Validierung
  IF employee_data ist leer:
    RETURN error("Mitarbeiter-Daten fehlen")
  IF name ist leer:
    RETURN error("Name ist erforderlich")
  IF email ist leer:
    RETURN error("Email ist erforderlich")
  IF email ist nicht valide:
    RETURN error("Email-Format ungültig")

  # Prüfung auf Duplikate
  existing_employee = FIND employee mit email IN database
  IF existing_employee existiert:
    RETURN error("Email bereits vergeben")

  # Mitarbeiter erstellen
  TRY:
    employee = CREATE new employee mit employee_data
    employee.created_at = aktuelle_zeit
    SAVE employee TO database
    RETURN success("Mitarbeiter erstellt", employee_data)
  CATCH database_error:
    RETURN error("Fehler beim Speichern")
```

**Siehst du?** Alle wichtigen Aspekte durchdacht, bevor wir programmieren!

---

# Pseudocode: Alle Mitarbeiter auflisten

## Einfache Aktionen richtig durchdenken

```
FUNCTION get_all_employees():

  # Daten aus Datenbank laden
  TRY:
    employees = LOAD all employees FROM database
  CATCH database_error:
    RETURN error("Fehler beim Laden der Mitarbeiter")

  # Vertrauliche Daten entfernen
  safe_employees = []
  FOR EACH employee IN employees:
    safe_employee = {
      id: employee.id,
      name: employee.name,
      email: employee.email,
      position: employee.position,
      start_date: employee.start_date
      # Gehalt bewusst NICHT zurückgeben!
    }
    ADD safe_employee TO safe_employees

  RETURN success("Mitarbeiter geladen", safe_employees)
```

**Wichtig:** Auch bei "einfachen" Aktionen an Sicherheit und Datenschutz denken!

---

# Pseudocode: Mitarbeiter aktualisieren

## Die komplexeste Operation richtig planen

```
FUNCTION update_employee(employee_id, update_data):

  # Input Validierung
  IF employee_id ist leer:
    RETURN error("Mitarbeiter-ID fehlt")
  IF update_data ist leer:
    RETURN error("Keine Update-Daten vorhanden")

  # Mitarbeiter laden
  TRY:
    employee = FIND employee mit employee_id IN database
  CATCH database_error:
    RETURN error("Fehler beim Laden")

  IF employee nicht gefunden:
    RETURN error("Mitarbeiter existiert nicht")

  # Email-Duplikat prüfen (falls Email geändert wird)
  IF update_data enthält neue_email:
    IF neue_email ist nicht valide:
      RETURN error("Email-Format ungültig")
    existing = FIND employee mit neue_email IN database
    IF existing existiert UND existing.id ist nicht employee_id:
```

```
      RETURN error("Email bereits vergeben")

  # Update durchführen
  TRY:
    UPDATE employee mit update_data IN database
    employee.updated_at = aktuelle_zeit
    RETURN success("Mitarbeiter aktualisiert", employee_data)
  CATCH database_error:
    RETURN error("Fehler beim Aktualisieren")
```

---

# Pseudocode: Mitarbeiter löschen

## Auch Löschen will gut durchdacht sein

```
FUNCTION delete_employee(employee_id):

  # Input Validierung
  IF employee_id ist leer:
    RETURN error("Mitarbeiter-ID fehlt")
  IF employee_id ist nicht numerisch:
    RETURN error("Ungültige ID")

  # Mitarbeiter suchen
  TRY:
    employee = FIND employee mit employee_id IN database
  CATCH database_error:
    RETURN error("Fehler beim Suchen")

  IF employee nicht gefunden:
    RETURN error("Mitarbeiter existiert nicht")

  # Löschen durchführen
  TRY:
    DELETE employee FROM database
    RETURN success("Mitarbeiter gelöscht")
  CATCH database_error:
    RETURN error("Fehler beim Löschen")
```

**Wichtig:** Auch beim Löschen alle Fälle durchdenken - was wenn der Mitarbeiter nicht existiert?

---

# Alle Pseudocode-Operationen im Überblick

## CRUD = Create, Read, Update, Delete

### Unsere 5 Hauptoperationen:

1. **CREATE** employee → POST mit Validierung und Duplikat-Check
2. **READ ALL** employees → GET mit Datenschutz (kein Gehalt)
3. **READ ONE** employee → GET mit Existenz-Check
4. **UPDATE** employee → PUT mit Validierung und Duplikat-Check
5. **DELETE** employee → DELETE mit Existenz-Check

### Gemeinsame Patterns in allem:

- **Input-Validierung** immer zuerst

- **Database-Errors** immer abfangen
- **Klare Fehlermeldungen** für bessere UX
- **Try-Catch** für alle DB-Operationen

**Jetzt haben wir einen kompletten Plan - Zeit für die Umsetzung!**

---

slideType: section

# Docker Compose Setup

---

# Warum Docker Compose?

## Die einfachste Art eine Datenbank zu starten

### Das Problem ohne Docker:

```
# PostgreSQL installieren... kompliziert!
sudo apt-get install postgresql
sudo -u postgres createdb employees_db
sudo -u postgres createuser --interactive
# Konfiguration, Passwörter, Ports...
# 1 Stunde später: "Es funktioniert immer noch nicht!"
```

### Die Lösung mit Docker Compose:

```
# docker-compose.yml
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: employees_db
      POSTGRES_PASSWORD: mypassword
    ports:
      - "5432:5432"
```

```
# Ein einziger Befehl:
docker-compose up -d
# Fertig! 🎉
```

**Vorteile:** Funktioniert auf jedem System, einfach zu löschen, keine Systemverschmutzung!

---

# Docker Compose einrichten

## Schritt-für-Schritt Setup für Anfänger

### Schritt 1: Docker Desktop installieren

- Gehe zu https://docker.com/desktop

- Downloade für dein System (Windows/Mac/Linux)
- Installiere und starte Docker Desktop

### Schritt 2: Projekt-Ordner erstellen

```
mkdir employee-api
cd employee-api
```

### Schritt 3: `docker-compose.yml` erstellen

```yaml
version: '3.8'

services:
  # PostgreSQL Datenbank
  db:
    image: postgres:15
    container_name: employee_db
    environment:
      POSTGRES_DB: employees_db
      POSTGRES_USER: api_user
      POSTGRES_PASSWORD: secure_password_123
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    restart: unless-stopped

volumes:
  postgres_data:
```

### Schritt 4: Datenbank starten

```
docker-compose up -d
```

**Das war's!** Deine PostgreSQL-Datenbank läuft jetzt.

---

# Datenbank prüfen und verstehen

## So siehst du ob alles funktioniert

### Schritt 5: Datenbank-Verbindung testen

```
# Datenbank-Container inspizieren
docker-compose ps
# Sollte zeigen: employee_db ... Up

# In die Datenbank verbinden (zum Testen)
docker exec -it employee_db psql -U api_user -d employees_db
```

### Wichtig: Wie entsteht das Schema?

```
-- Zu diesem Zeitpunkt ist die DB noch LEER!
-- Keine Tabellen, keine Struktur - nur eine leere Datenbank

-- Das passiert erst wenn wir SQLAlchemy starten:
-- Base.metadata.create_all(bind=engine)
```

```
-- ↑ Dieser Befehl erstellt alle Tabellen automatisch
```

### Was passiert beim ersten Start?

1. **Docker Container** startet → PostgreSQL läuft
2. **Datenbank "employees_db"** existiert → aber ist leer
3. **FastAPI startet** → SQLAlchemy verbindet sich
4. **Base.metadata.create_all()** → Tabellen werden erstellt
5. **Fertig!** Schema ist da, API kann arbeiten

### Vorteile dieser Methode:

- **Automatisch** - keine manuellen SQL-Scripts
- **Sicher** - Schema passt immer zum Code
- **Einfach** - für Entwicklung perfekt

---

slideType: section

# FastAPI Implementation

---

# Projekt-Setup

## Von der Planung zur Umsetzung

### Schritt 1: Virtual Environment

```
# Im employee-api Ordner
python -m venv venv

# Aktivieren (Windows)
venv\Scripts\activate

# Aktivieren (Mac/Linux)
source venv/bin/activate
```

### Schritt 2: Dependencies installieren

```
# Alle benötigten Pakete
pip install fastapi uvicorn sqlalchemy psycopg2-binary pydantic[email]

# Requirements speichern
pip freeze > requirements.txt
```

### Warum diese Dependencies?

- `fastapi` - unser Web-Framework
- `uvicorn` - ASGI Server (lädt FastAPI)
- `sqlalchemy` - Object-Relational Mapper (ORM)
- `psycopg2-binary` - PostgreSQL Treiber

- `pydantic[email]` - Datenvalidierung + Email-Validierung

**Warum pydantic[email]?**

```
# Ohne [email]: Nur string-Validierung
email: str  # Akzeptiert "invalid-email" ❌

# Mit [email]: Echte Email-Validierung
email: EmailStr  # Akzeptiert nur "user@domain.com" ✅
```

### Schritt 3: Projekt-Struktur

```
employee-api/
├── docker-compose.yml    # Datenbank
├── requirements.txt      # Python Dependencies
├── main.py              # FastAPI App
├── models.py            # Pydantic Models
├── database.py          # DB Connection
└── crud.py              # CRUD Operations
```

**Übersichtlich und professionell strukturiert!**

---

# Datenmodelle definieren

## Unser Pseudocode wird zu Pydantic

### Erstelle `models.py`:

```python
from pydantic import BaseModel, EmailStr, Field
from typing import Optional
from datetime import datetime

# Base Model mit gemeinsamen Feldern
class EmployeeBase(BaseModel):
    name: str = Field(min_length=2, max_length=100, description="Vollständiger Name")
    email: EmailStr = Field(description="Gültige Email-Adresse")
    position: Optional[str] = Field(None, max_length=100, description="Position/Rolle")
    start_date: Optional[datetime] = Field(None, description="Startdatum")

# Für die Erstellung (mit allen Feldern)
class EmployeeCreate(EmployeeBase):
    salary: Optional[float] = Field(None, ge=0, description="Gehalt (optional)")

# Für Updates (alle Felder optional)
class EmployeeUpdate(BaseModel):
    name: Optional[str] = Field(None, min_length=2, max_length=100)
    email: Optional[EmailStr] = None
    position: Optional[str] = Field(None, max_length=100)
    salary: Optional[float] = Field(None, ge=0)

# Für Responses (ohne Gehalt!)
class Employee(EmployeeBase):
    id: int
    created_at: datetime
    updated_at: Optional[datetime] = None

    class Config:
        from_attributes = True  # Für SQLAlchemy Kompatibilität
```

**Siehst du?** Unser Pseudocode wird zu echten Datenmodellen!

# Datenbank-Verbindung

## SQLAlchemy + Docker Compose

### Erstelle `database.py`:

```python
from sqlalchemy import create_engine, Column, Integer, String, Float, DateTime
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from datetime import datetime

# Database URL für unseren Docker Container
DATABASE_URL = "postgresql://api_user:secure_password_123@localhost:5432/employees_db"

# SQLAlchemy Setup
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

# Database Model (für SQLAlchemy)
class EmployeeDB(Base):
    __tablename__ = "employees"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)
    email = Column(String(255), unique=True, nullable=False, index=True)
    position = Column(String(100), nullable=True)
    start_date = Column(DateTime, nullable=True)
    salary = Column(Float, nullable=True)  # Vertraulich!
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, nullable=True)

# Tabellen erstellen
Base.metadata.create_all(bind=engine)

# Dependency für DB Sessions
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

# CRUD-Operationen

## Pseudocode wird zu echtem Python

### Erstelle `crud.py`:

```python
from sqlalchemy.orm import Session
from sqlalchemy.exc import IntegrityError
from database import EmployeeDB
from models import EmployeeCreate, EmployeeUpdate
from datetime import datetime

def create_employee(db: Session, employee: EmployeeCreate):
    """Mitarbeiter erstellen - folgt unserem Pseudocode!"""
```

```python
    # Prüfung auf Email-Duplikat
    existing = db.query(EmployeeDB).filter(EmployeeDB.email == employee.email).first()
    if existing:
        raise ValueError("Email bereits vergeben")

    # Neuen Mitarbeiter erstellen
    db_employee = EmployeeDB(
        name=employee.name,
        email=employee.email,
        position=employee.position,
        start_date=employee.start_date,
        salary=employee.salary,
        created_at=datetime.utcnow()
    )

    try:
        db.add(db_employee)
        db.commit()
        db.refresh(db_employee)
        return db_employee
    except IntegrityError:
        db.rollback()
        raise ValueError("Fehler beim Speichern")

def get_employee(db: Session, employee_id: int):
    """Einzelnen Mitarbeiter laden"""
    return db.query(EmployeeDB).filter(EmployeeDB.id == employee_id).first()

def get_employees(db: Session, skip: int = 0, limit: int = 100):
    """Alle Mitarbeiter laden"""
    return db.query(EmployeeDB).offset(skip).limit(limit).all()

def update_employee(db: Session, employee_id: int, employee_update: EmployeeUpdate):
    """Mitarbeiter aktualisieren - folgt unserem Pseudocode!"""

    # Mitarbeiter finden
    db_employee = db.query(EmployeeDB).filter(EmployeeDB.id == employee_id).first()
    if not db_employee:
        return None

    # Email-Duplikat prüfen (falls Email geändert wird)
    if employee_update.email and employee_update.email != db_employee.email:
        existing = db.query(EmployeeDB).filter(EmployeeDB.email == employee_update.email).first()
        if existing:
            raise ValueError("Email bereits vergeben")

    # Update durchführen
    update_data = employee_update.dict(exclude_unset=True)
    for key, value in update_data.items():
        setattr(db_employee, key, value)

    db_employee.updated_at = datetime.utcnow()

    try:
        db.commit()
        db.refresh(db_employee)
        return db_employee
    except IntegrityError:
        db.rollback()
        raise ValueError("Fehler beim Aktualisieren")

def delete_employee(db: Session, employee_id: int):
    """Mitarbeiter löschen"""
    db_employee = db.query(EmployeeDB).filter(EmployeeDB.id == employee_id).first()
    if db_employee:
        db.delete(db_employee)
        db.commit()
        return True
    return False
```

# FastAPI Endpoints

## Pseudocode wird zu REST API

### Erstelle `main.py`:

```python
from fastapi import FastAPI, Depends, HTTPException, status
from sqlalchemy.orm import Session
from typing import List

from database import get_db
from models import Employee, EmployeeCreate, EmployeeUpdate
import crud

app = FastAPI(
    title="Employee Management API",
    description="Mitarbeiter-Verwaltung - Von Pseudocode zu produktiver API",
    version="1.0.0"
)

# Health Check
@app.get("/health")
def health_check():
    return {"status": "healthy", "service": "employee-api"}

# POST /employees - Mitarbeiter erstellen
@app.post("/employees", response_model=Employee, status_code=status.HTTP_201_CREATED)
def create_employee(employee: EmployeeCreate, db: Session = Depends(get_db)):
    """
    Neuen Mitarbeiter erstellen - folgt exakt unserem Pseudocode!

    - Validiert alle Eingabedaten automatisch durch Pydantic
    - Prüft auf Email-Duplikate
    - Speichert in der Datenbank
    - Gibt Mitarbeiter ohne Gehalt zurück (Datenschutz!)
    """
    try:
        db_employee = crud.create_employee(db=db, employee=employee)
        return db_employee
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))

# GET /employees - Alle Mitarbeiter auflisten
@app.get("/employees", response_model=List[Employee])
def get_employees(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    """
    Alle Mitarbeiter auflisten

    - Paginierung mit skip/limit
    - Gehalt wird automatisch ausgeschlossen (Employee Model hat kein salary)
    """
    employees = crud.get_employees(db, skip=skip, limit=limit)
    return employees

# GET /employees/{id} - Einzelnen Mitarbeiter anzeigen
@app.get("/employees/{employee_id}", response_model=Employee)
def get_employee(employee_id: int, db: Session = Depends(get_db)):
    """Einzelnen Mitarbeiter anzeigen"""
    db_employee = crud.get_employee(db, employee_id=employee_id)
    if db_employee is None:
        raise HTTPException(
            status_code=404,
            detail=f"Mitarbeiter mit ID {employee_id} nicht gefunden"
        )
    return db_employee
```

```python
# PUT /employees/{id} - Mitarbeiter aktualisieren
@app.put("/employees/{employee_id}", response_model=Employee)
def update_employee(employee_id: int, employee: EmployeeUpdate, db: Session = Depends(get_db)):
    """
    Mitarbeiter aktualisieren - folgt unserem Pseudocode!

    - Nur gesetzte Felder werden aktualisiert
    - Email-Duplikat-Prüfung
    - Automatisches updated_at Timestamp
    """
    try:
        db_employee = crud.update_employee(db, employee_id=employee_id, employee_update=employee)
        if db_employee is None:
            raise HTTPException(
                status_code=404,
                detail=f"Mitarbeiter mit ID {employee_id} nicht gefunden"
            )
        return db_employee
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))

# DELETE /employees/{id} - Mitarbeiter löschen
@app.delete("/employees/{employee_id}")
def delete_employee(employee_id: int, db: Session = Depends(get_db)):
    """Mitarbeiter löschen"""
    success = crud.delete_employee(db, employee_id=employee_id)
    if not success:
        raise HTTPException(
            status_code=404,
            detail=f"Mitarbeiter mit ID {employee_id} nicht gefunden"
        )
    return {"message": f"Mitarbeiter mit ID {employee_id} erfolgreich gelöscht"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

---

# API starten und testen

## Unser geplante API zum Leben erwecken

### Schritt 1: Docker Datenbank starten

```bash
# Falls noch nicht gestartet
docker-compose up -d

# Prüfen ob läuft
docker-compose ps
```

### Schritt 2: FastAPI starten

```bash
# Virtual Environment aktivieren
source venv/bin/activate  # Mac/Linux
# venv\Scripts\activate   # Windows

# Server starten
uvicorn main:app --reload
```

### Schritt 3: Automatische Dokumentation anschauen

- Gehe zu: http://localhost:8000/docs
- **WOW!** Komplette interaktive Dokumentation
- Alle Endpoints sind da
- Du kannst direkt testen!

### Schritt 4: Health Check testen

- Gehe zu: http://localhost:8000/health
- Sollte zurückgeben: `{"status": "healthy", "service": "employee-api"}`

**Geschafft! Unsere geplante API läuft!**

---

# Troubleshooting: Häufige Anfängerprobleme

## Wenn etwas nicht funktioniert - keine Panik!

### Problem 1: Docker Container läuft nicht

```
# Fehler: "connection refused" oder "server not found"
# Lösung: Docker Status prüfen
docker-compose ps
# Wenn "Exit 1" oder "Down": Container neustarten
docker-compose down
docker-compose up -d
```

### Problem 2: Port bereits belegt

```
# Fehler: "Port 5432 already in use"
# Lösung 1: Anderen Port verwenden
# In docker-compose.yml ändern:
ports:
  - "5433:5432"  # Statt 5432:5432

# Lösung 2: Bestehenden Service stoppen
sudo lsof -i :5432  # Zeigt was Port 5432 nutzt
```

### Problem 3: Datenbank-Verbindung schlägt fehl

```
# Fehler: "could not connect to server"
# Check 1: Container läuft?
docker-compose ps

# Check 2: Richtige Credentials?
DATABASE_URL = "postgresql://api_user:secure_password_123@localhost:5432/employees_db"
# ↑ User, Password, Port müssen exakt zu docker-compose.yml passen

# Check 3: PostgreSQL Treiber installiert?
pip install psycopg2-binary
```

### Problem 4: FastAPI startet nicht

```
# Fehler: "ModuleNotFoundError"
# Lösung: Virtual Environment aktivieren
source venv/bin/activate  # Mac/Linux
venv\Scripts\activate     # Windows
```

```
# Dann Dependencies neu installieren
pip install fastapi uvicorn sqlalchemy psycopg2-binary pydantic[email]
```

### Problem 5: API-Endpoints funktionieren nicht

```
# Fehler: 500 Internal Server Error
# Check 1: Tabellen erstellt?
# Base.metadata.create_all(bind=engine) muss ausgeführt werden

# Check 2: Import-Fehler?
# Alle Imports prüfen:
from database import get_db
from models import Employee, EmployeeCreate, EmployeeUpdate
import crud
```

### Quick-Fix Checkliste:

1. ✅ **Docker läuft?** → `docker-compose ps`
2. ✅ **Virtual Env aktiv?** → Prompt sollte `(venv)` zeigen
3. ✅ **Dependencies installiert?** → `pip list` zeigt fastapi
4. ✅ **Ports frei?** → 5432 und 8000 nicht belegt
5. ✅ **Credentials richtig?** → docker-compose.yml vs. DATABASE_URL

**Wenn alles stimmt: API läuft in 2 Minuten!**

---

slideType: section

# Postman Testing

---

# Postman Collection erstellen

## Systematische Tests für unsere geplante API

### Schritt 1: Collection Setup

1. **Neue Collection:** "Employee Management API"
2. **Environment Variable:**

   ```
   base_url = http://localhost:8000
   ```

### Schritt 2: Alle Requests erstellen

#### 1. Health Check

```
GET {{base_url}}/health
Expected: 200 OK
{
  "status": "healthy",
  "service": "employee-api"
}
```

### 2. Mitarbeiter erstellen

```
POST {{base_url}}/employees
Content-Type: application/json

{
  "name": "Max Mustermann",
  "email": "max.mustermann@firma.de",
  "position": "Software Developer",
  "start_date": "2024-01-15T09:00:00"
}

Expected: 201 Created + Employee-Objekt
```

### 3. Alle Mitarbeiter auflisten

```
GET {{base_url}}/employees

Expected: 200 OK + Array von Mitarbeitern
```

### 4. Einzelnen Mitarbeiter anzeigen

```
GET {{base_url}}/employees/1

Expected: 200 OK + Employee-Objekt
```

### 5. Mitarbeiter aktualisieren

```
PUT {{base_url}}/employees/1
Content-Type: application/json

{
  "position": "Senior Software Developer"
}

Expected: 200 OK + Aktualisiertes Employee-Objekt
```

### 6. Mitarbeiter löschen

```
DELETE {{base_url}}/employees/1

Expected: 200 OK + Lösch-Bestätigung
```

---

# Automatisierte Tests in Postman

## Wie Profis ihre APIs testen

### Test-Scripts für jeden Request:

#### Mitarbeiter erstellen - Tests:

```
// Status Code prüfen
pm.test("Status code is 201", function () {
```

```
    pm.response.to.have.status(201);
});

// Response-Struktur prüfen
pm.test("Employee created successfully", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('id');
    pm.expect(jsonData).to.have.property('name');
    pm.expect(jsonData).to.have.property('email');
    pm.expect(jsonData.name).to.eql("Max Mustermann");
});

// Employee ID für weitere Tests speichern
pm.test("Save employee ID", function () {
    var jsonData = pm.response.json();
    pm.environment.set("employee_id", jsonData.id);
});

// Response Zeit prüfen
pm.test("Response time is less than 1000ms", function () {
    pm.expect(pm.response.responseTime).to.be.below(1000);
});
```

**Error-Tests hinzufügen:**

```
// Test mit ungültiger Email
POST {{base_url}}/employees
{
  "name": "Test User",
  "email": "ungueltige-email",
  "position": "Tester"
}

// Erwartete Response: 422 Validation Error
pm.test("Invalid email rejected", function () {
    pm.response.to.have.status(422);
    var jsonData = pm.response.json();
    pm.expect(jsonData.detail[0].msg).to.include("valid email");
});
```

### Collection Runner nutzen:

- Alle Tests automatisch durchlaufen lassen
- Verschiedene Environments testen (dev, staging, prod)
- Reports generieren

---

slideType: section

# Workshop: Live Implementation

---

# Workshop-Zeit: Live-Coding

## Jetzt bauen wir gemeinsam unsere geplante API!

### Was wir gemeinsam machen werden:

**Phase 1: Umgebung vorbereiten (10 Min)**

1. **Docker Compose starten** - Datenbank mit einem Befehl
2. **Virtual Environment setup** - Python-Umgebung isolieren
3. **Dependencies installieren** - FastAPI und alle Tools

**Phase 2: Modelle definieren (15 Min)**

1. **Pydantic Models erstellen** - unser Pseudocode wird zu Python
2. **Validierungsregeln implementieren** - automatische Eingabeprüfung
3. **SQLAlchemy Models** - Datenbank-Schema definieren

**Phase 3: CRUD implementieren (20 Min)**

1. **Database Connection** - PostgreSQL anbinden
2. **Create-Operation** - folgt unserem Pseudocode!
3. **Read-Operationen** - einzeln und alle
4. **Update & Delete** - vollständiges CRUD

**Phase 4: FastAPI Endpoints (20 Min)**

1. **API-Struktur erstellen** - REST-konforme Endpoints
2. **Error-Handling** - professionelle Fehlerbehandlung
3. **Automatische Dokumentation** - FastAPI's Superkraft
4. **Live-Testing** - in der generierten Docs

**Phase 5: Postman Collection (10 Min)**

1. **Collection erstellen** - alle Endpoints organisiert
2. **Tests automatisieren** - JavaScript-basierte Validierung
3. **Error-Cases testen** - auch Fehlerfälle abdecken

### Am Ende habt ihr:

- Eine vollständige, geplante REST API
- Docker-basierte Entwicklungsumgebung
- Automatische API-Dokumentation
- Professionelle Postman-Tests
- Code der eurem Pseudocode folgt!

**Los geht's - ich zeige jeden Schritt live!**

---

# Phase 1: Umgebung Live Setup

## Docker Compose Magic

### Live Demo: Ein-Befehl-Datenbank

```
# Neues Projekt erstellen
mkdir employee-api-live
cd employee-api-live
```

```
# Docker Compose File erstellen (live coding)
nano docker-compose.yml
```

### Live: docker-compose.yml erstellen

```
version: '3.8'
services:
  db:
    image: postgres:15
    container_name: workshop_db
    environment:
      POSTGRES_DB: employees_db
      POSTGRES_USER: workshop_user
      POSTGRES_PASSWORD: workshop_pass
    ports:
      - "5432:5432"
    volumes:
      - workshop_data:/var/lib/postgresql/data
volumes:
  workshop_data:
```

```
# Datenbank starten - ein einziger Befehl!
docker-compose up -d

# Prüfen ob läuft
docker-compose ps
# Status: Up - PERFEKT!
```

### Python Environment Setup

```
python -m venv workshop-env
source workshop-env/bin/activate  # Mac/Linux
pip install fastapi uvicorn sqlalchemy psycopg2-binary pydantic[email]
pip freeze > requirements.txt
```

**→ In 3 Minuten: Komplette Entwicklungsumgebung läuft!**

---

# Phase 2: Modelle Live Coding

## Pseudocode wird zu Python

### Live: `models.py` erstellen

```
# Ich zeige jeden Schritt live...
from pydantic import BaseModel, EmailStr, Field
from typing import Optional
from datetime import datetime

# Unser Pseudocode sagte: "Employee hat Name, Email, Position..."
class EmployeeBase(BaseModel):
    name: str = Field(min_length=2, max_length=100)
    email: EmailStr
    position: Optional[str] = Field(None, max_length=100)

# Für Create - alles was wir brauchen
class EmployeeCreate(EmployeeBase):
    salary: Optional[float] = Field(None, ge=0)  # Mindestens 0

# Für Response - OHNE Gehalt (Datenschutz!)
```

```python
class Employee(EmployeeBase):
    id: int
    created_at: datetime
    updated_at: Optional[datetime] = None

    class Config:
        from_attributes = True
```

### Live: `database.py` erstellen

```python
# Folgt unserem Plan: PostgreSQL via Docker
from sqlalchemy import create_engine, Column, Integer, String, Float, DateTime
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from datetime import datetime

# Docker Container Connection
DATABASE_URL = "postgresql://workshop_user:workshop_pass@localhost:5432/employees_db"

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

class EmployeeDB(Base):
    __tablename__ = "employees"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)
    email = Column(String(255), unique=True, nullable=False)
    position = Column(String(100))
    salary = Column(Float)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime)

# Tabellen erstellen
Base.metadata.create_all(bind=engine)
```

**→ Unser Pseudocode wird zu echten Datenmodellen!**

---

# Phase 3: CRUD Live Implementation

## Pseudocode wird zu Datenbankoperationen

### Live: `crud.py` - Create Employee

```python
# Folge exakt unserem Pseudocode!
from sqlalchemy.orm import Session
from database import EmployeeDB
from models import EmployeeCreate
from datetime import datetime

def create_employee(db: Session, employee: EmployeeCreate):
    """
    FUNCTION create_employee(employee_data):
      IF email bereits existiert:
        RETURN error("Email bereits vergeben")
      SAVE employee TO database
    """

    # 1. Prüfen ob Email existiert (wie im Pseudocode!)
    existing = db.query(EmployeeDB).filter(EmployeeDB.email == employee.email).first()
    if existing:
```

```
        raise ValueError("Email bereits vergeben")

    # 2. Neuen Employee erstellen
    db_employee = EmployeeDB(
        name=employee.name,
        email=employee.email,
        position=employee.position,
        salary=employee.salary,
        created_at=datetime.utcnow()
    )

    # 3. Speichern
    db.add(db_employee)
    db.commit()
    db.refresh(db_employee)

    return db_employee

# Live implementieren: get_employees, get_employee, update_employee, delete_employee
```

**→ Ich zeige live wie jeder Pseudocode-Block zu echtem Python wird!**

---

# Phase 4: FastAPI Endpoints Live

## API-Planung wird zu REST Endpoints

### Live: `main.py` - Employee Creation

```
from fastapi import FastAPI, Depends, HTTPException, status
from sqlalchemy.orm import Session
from database import get_db
from models import Employee, EmployeeCreate
import crud

app = FastAPI(title="Workshop Employee API")

@app.post("/employees", response_model=Employee, status_code=201)
def create_employee(employee: EmployeeCreate, db: Session = Depends(get_db)):
    """
    Unser Pseudocode wird zu REST API:
    POST /employees → create_employee(employee_data)
    """
    try:
        # Folgt exakt unserem CRUD-Pseudocode
        db_employee = crud.create_employee(db=db, employee=employee)
        return db_employee
    except ValueError as e:
        # Error-Handling wie geplant
        raise HTTPException(status_code=400, detail=str(e))

# Live implementieren: GET /employees, GET /employees/{id}, PUT, DELETE
```

### Live-Test in der automatischen Dokumentation:

```
# Server starten
uvicorn main:app --reload

# Browser öffnen: http://localhost:8000/docs
# → WOW! Komplette interaktive API-Dokumentation!
```

**→ Live-Demo: Von der Planung zur funktionsfähigen API in Minuten!**

# Zusammenfassung: Pseudocode-Driven Development

## Die wichtigsten Erkenntnisse zum Mitnehmen

### Warum Pseudocode-First so mächtig ist:

- **Weniger Bugs** - Logik ist vor Implementierung durchdacht
- **Klarere Struktur** - Code folgt deinem Plan
- **Bessere Kommunikation** - Team versteht deinen Ansatz
- **Schnellere Entwicklung** - weniger "Hin und Her"

### FastAPI + Docker = Perfekte Kombination:

- **FastAPI** - entspricht natürlich unserem strukturierten Denken
- **Docker Compose** - Entwicklungsumgebung in Sekunden
- **Automatische Dokumentation** - API erklärt sich selbst
- **Type Safety** - Pydantic verhindert viele Fehler

### Von User Story zu produkiver API:

1. **User Story analysieren** - was genau wird gebraucht?
2. **Endpoints planen** - REST-konforme URL-Struktur
3. **Pseudocode schreiben** - Logik vor Syntax
4. **Modelle definieren** - Pydantic macht Validierung automatisch
5. **CRUD implementieren** - folge dem Pseudocode
6. **API-Endpoints** - FastAPI macht es einfach
7. **Testen mit Postman** - systematische Qualitätssicherung

### Das Wichtigste:

> "Geplante APIs sind bessere APIs. Pseudocode ist dein Bauplan für erfolgreiche Software-Entwicklung!"

**Du hast heute gelernt wie echte Profis APIs entwickeln - mit System und Plan!**

---

slideType: section

# Feedback & Fragen

---

# Noch Fragen?

## Jetzt ist Zeit für eure Fragen und euer Feedback!

### Wie fandet ihr die Präsentation?

**Bitte gebt uns kurzes Feedback im Chat:**

- 👍 **Daumen hoch** - Der Pseudocode-Ansatz ist genial!
- 👎 **Daumen runter** - Könnte besser sein
- ❓ **Fragezeichen** - Ich habe noch Fragen
- 💡 **Glühbirne** - Ich habe eine Idee dazu

### Typische Fragen die oft kommen:

- "Ist Pseudocode nicht Zeitverschwendung?"
- "Wie detailliert soll Pseudocode sein?"
- "Kann ich das auch für Frontend-Entwicklung nutzen?"
- "Docker Compose vs. lokale PostgreSQL-Installation?"
- "Wie deploye ich FastAPI mit Docker in Production?"

### Eure Fragen:

*[Platz für Live-Fragen aus dem Chat/der Runde]*

**Traut euch - es gibt keine dummen Fragen, nur ungestellte!**

---

slideType: section

# Vielen Dank!

---

# Vielen Dank für eure Aufmerksamkeit!

## Ihr habt heute etwas Großartiges gelernt

### Was ihr heute gemeistert habt:

- **Pseudocode-Driven Development** - planen vor programmieren
- **FastAPI von der Pike auf** - moderne Python-APIs
- **Docker Compose** - professionelle Entwicklungsumgebung
- **REST API Best Practices** - wie es richtig gemacht wird

### Nächste Schritte:

1. **Erweitert eure Workshop-API** mit weiteren Endpoints
2. **Verwendet Pseudocode** für euer nächstes Projekt
3. **Teilt euer Wissen** mit anderen Entwicklern
4. **Dokumentiert eure APIs** - FastAPI macht es einfach

### Kontakt & Ressourcen:

- **Fragen später?** Meldet euch gerne bei uns
- **Weitere Python-Schulungen?** Authentication, Testing, Deployment
- **Team-Workshops?** Gerne kommen wir auch zu euch

### Das Wichtigste zum Schluss:

> "Jede große Software fängt mit einem guten Plan an. Ihr habt heute gelernt wie man diesen Plan erstellt und umsetzt!"

**Viel Spaß beim Planen und Bauen eurer eigenen APIs! 🚀**